CSC738 Advanced Machine Learning · Topic 3 · Week 4

Supervised Learning I: Classification Methods

Classical classifiers, ensembles, cross-validation and hyperparameter tuning

- **Objectives**
  - Explain how Logistic Regression, Naïve Bayes, KNN, SVM and Decision Trees classify data
  - Compare bagging and boosting ensembles: Random Forest, AdaBoost, Gradient Boosting, XGBoost
  - Choose a suitable classifier for a given dataset
  - Compare models fairly with a baseline and stratified cross-validation
  - Tune hyperparameters with grid or random search inside a Pipeline
  - Evaluate the final model once on a held-out test set

What is Machine Learning (ML)?

- ML is a method of data analysis that automates analytical model building. It is a branch of artificial intelligence based on the idea that systems can learn from data, identify patterns and make decisions with minimal human intervention.
- ML is important because now we can quickly and automatically produce models that can analyze bigger, more complex data and deliver faster, more accurate results – even on a very large scale.
- And by building precise models, an organization has a better chance of identifying profitable opportunities – or avoiding unknown risks.

- Did you know?
  - In machine learning, a target is called a label while in statistics, a target is called a dependent variable.
  - A variable in statistics is called a feature in machine learning.
  - A transformation in statistics is called feature creation in machine learning
- What's required to create good machine learning systems?
  - Data preparation capabilities.
  - Algorithms – basic and advanced.
  - Automation and iterative processes.
  - Scalability.
  - Ensemble modeling.

Who is using ML?











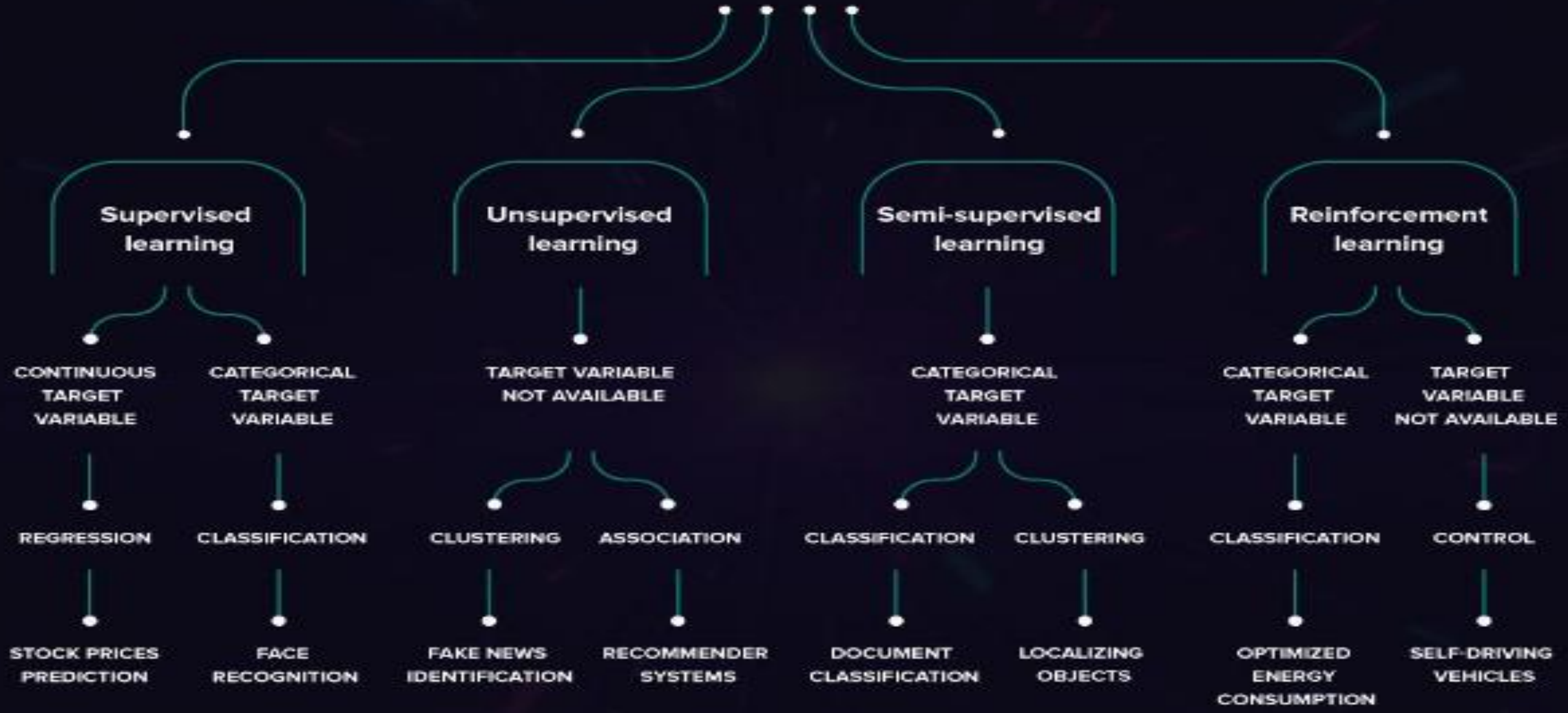


Machine Learning vs Data Mining

- **Similarities**
  - Both are analytics processes
  - Both are good at pattern recognition
  - Both are about learning from data so that we can improve decision making
  - Both require large amounts of data to be accurate

- **Differences**
  - DM is a more manual process that relies on human intervention and decision making. But, with ML, once the initial rules are in place, the process of extracting information and 'learning' and refining is automatic and takes place without human intervention.
  - DM is used on an existing dataset (like a data warehouse) to find patterns. ML is trained on a 'training' data set, which teaches the computer how to make sense of data, and then to make predictions about new data sets.

MACHINE LEARNING TYPES



This week in the workflow

1 **Baseline**

2 **Candidate models**

3 **Cross-validation**

4 **Tune**

5 **Test once**

Steps 6–7 of the ML workflow: choose candidate classifiers, compare them fairly with cross-validation, tune the best, then test once.

Data preparation and features (Weeks 2–3) come first; regression and forecasting follow in Week 5.

Part 1

Classical classifiers

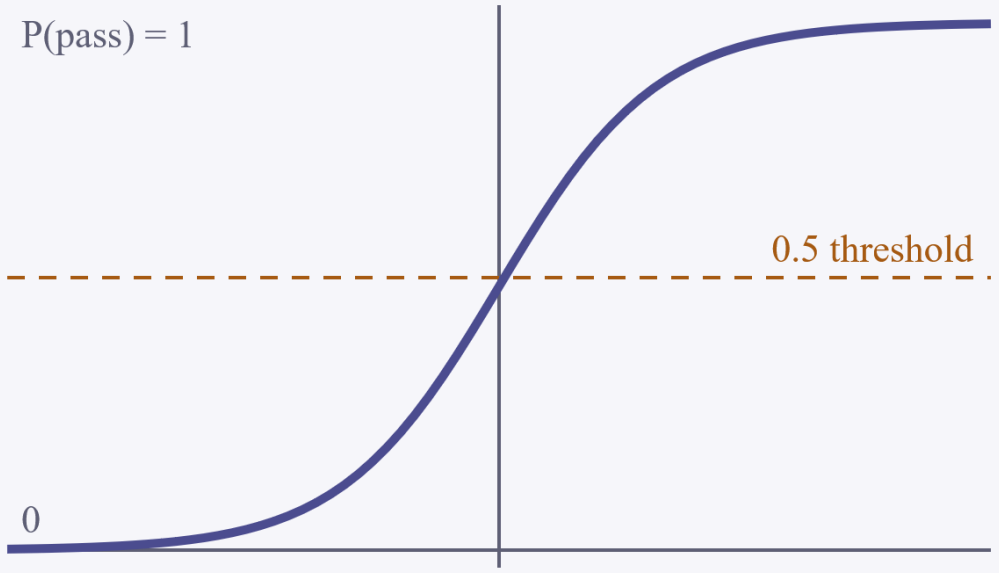
Logistic Regression, Naïve Bayes, KNN, SVM, Decision Trees

Logistic Regression

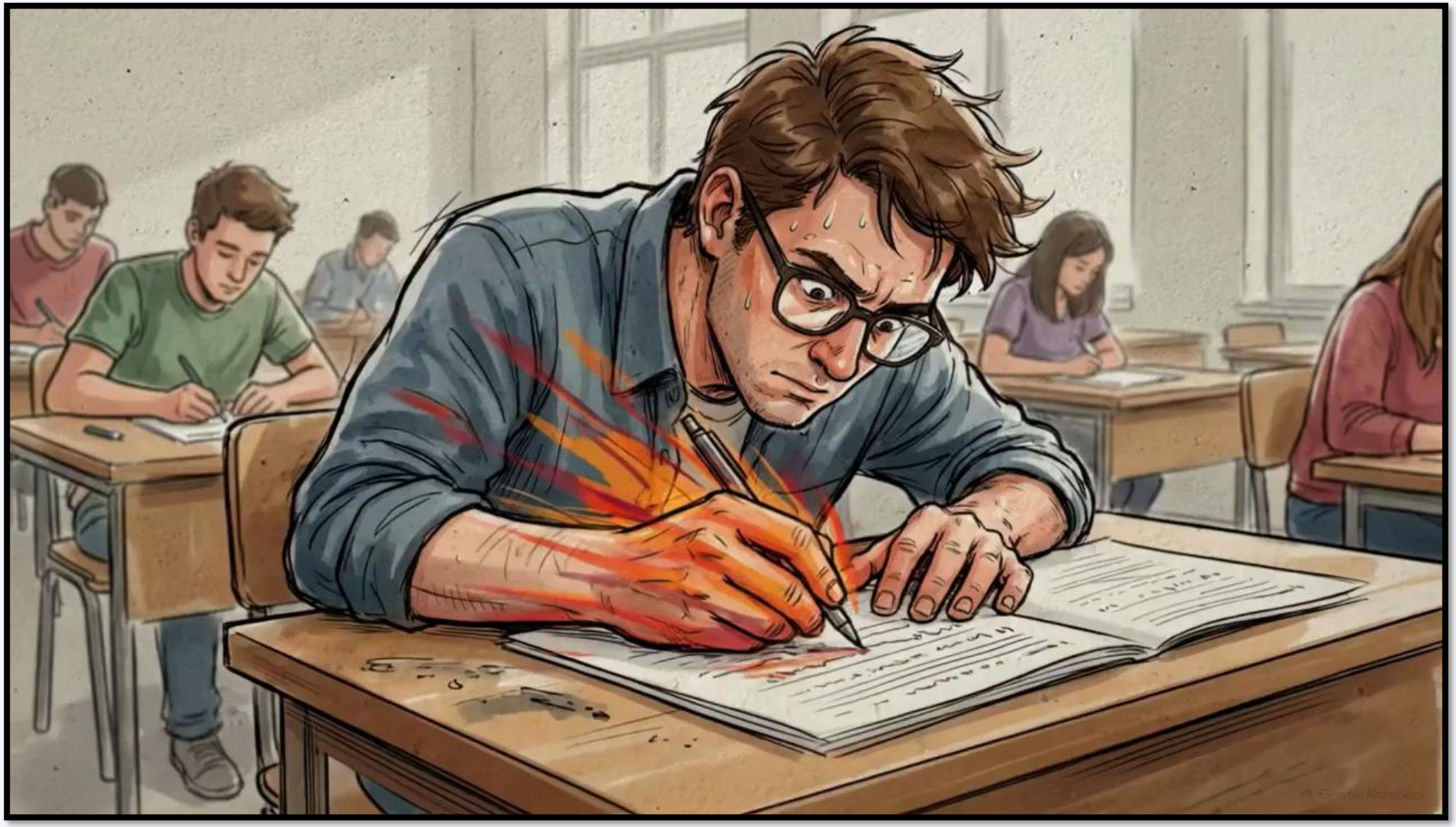
Combines the features into a weighted score, then squeezes it through the **sigmoid** into a probability between 0 and 1.

$$P(y = 1) = 1 / (1 + e^{-(w \cdot x + b)})$$

- Predict class 1 when the probability ≥ 0.5 (the threshold can be moved)
- Weights show each feature's direction and strength: easy to explain
- Draws a straight (linear) boundary; needs scaling
- Key hyperparameter: C (smaller C = stronger regularisation)



Logistic Regression Process



Naïve Bayes

Uses Bayes' rule to pick the most probable class given the features:

$$P(\text{class} \mid x) \propto P(\text{class}) \times P(x_1 \mid \text{class}) \times \ldots \times P(x_n \mid \text{class})$$

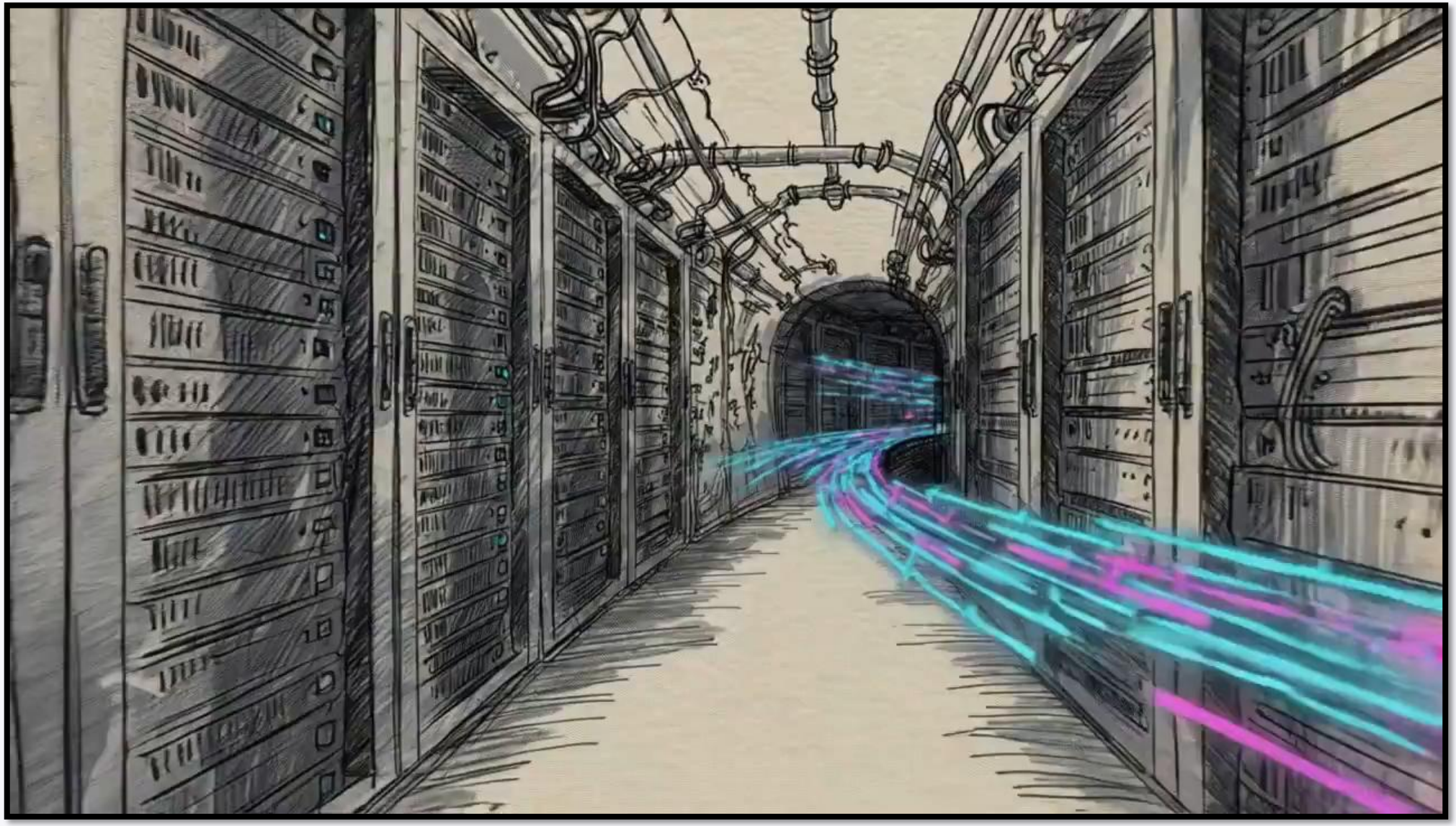
"Naïve": assumes features are independent given the class. Rarely true, but it often works well anyway.

Example: spam filter

If "free", "win" and "click" appear, multiply their probabilities under spam and under not-spam, and pick the larger.

Variant	Use for
Gaussian	Continuous features
Multinomial	Word counts, TF-IDF
Bernoulli	Binary (yes/no) features

Naïve Bayes Process

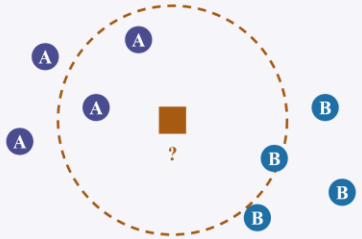


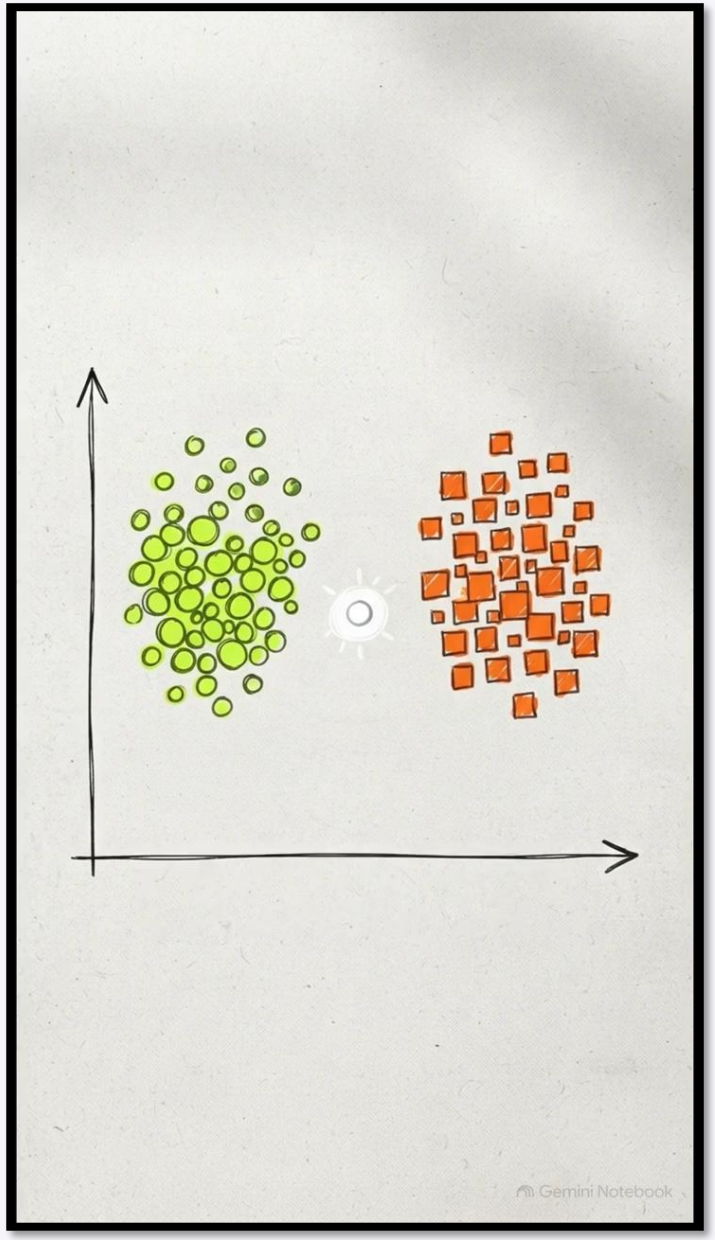
K-Nearest Neighbours (KNN)

1. Measure the distance (e.g. Euclidean) from the new point to every training point
2. Take the k closest neighbours
3. Predict the majority class among them

Choice	Effect
Small k (e.g. 1)	Follows noise: overfits
Large k	Too smooth: underfits
Odd k	Avoids ties in binary problems

Distance-based, so always scale features (Week 2 lab: 0.68 → 0.78).



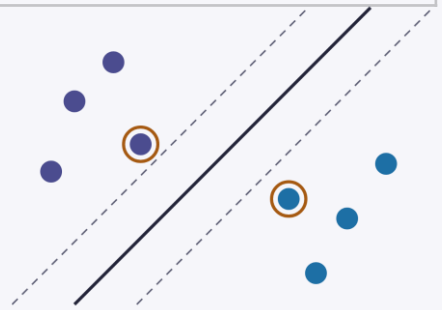


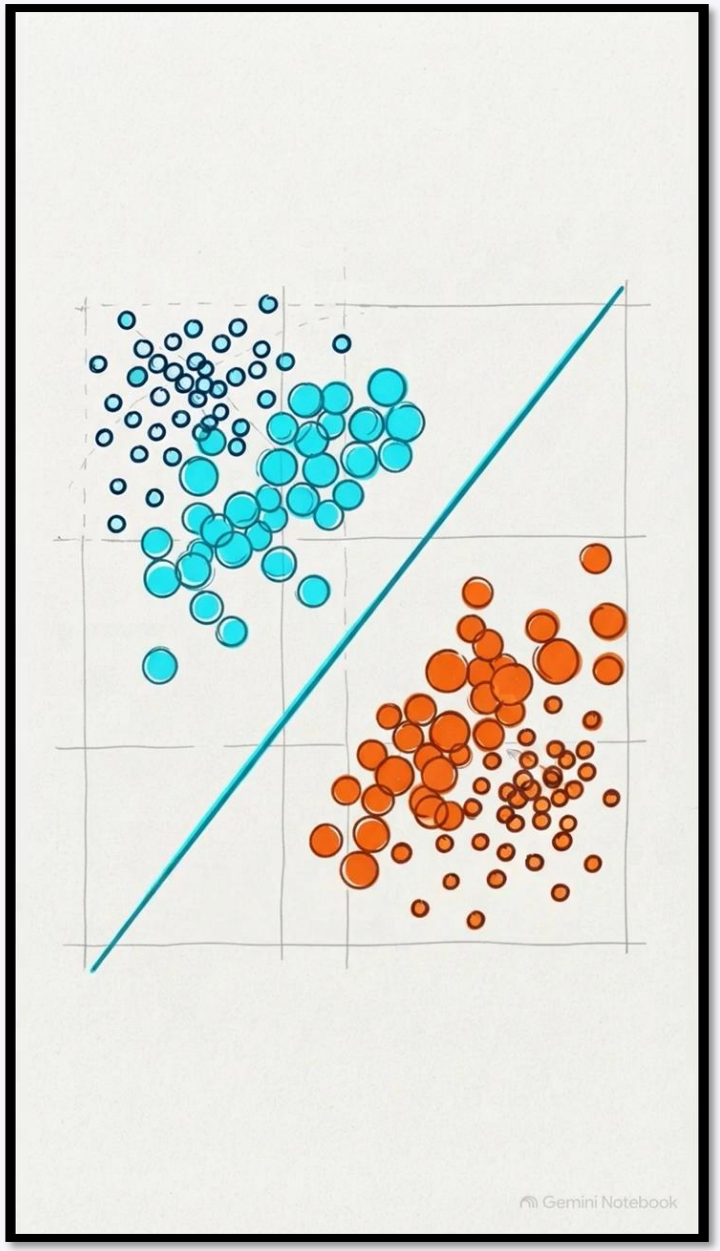
Support Vector Machine (SVM)

Finds the boundary (hyperplane) that separates the classes with the **widest margin**.

- Support vectors (orange rings): the points closest to the boundary; they alone define it
- A wide margin generalises better to new data
- Soft margin: allows a few errors; controlled by C

C	Effect
Small	Wider margin, more errors allowed: may underfit
Large	Narrow margin, few errors allowed: may overfit





SVM kernels

When classes cannot be split by a straight line, a **kernel** lets SVM draw curved boundaries.

Kernel	Formula	Use when
Linear	$K(x, x_i) = x \cdot x_i$	Many features (e.g. TF-IDF text); roughly linear data
Polynomial	$K(x, x_i) = (1 + x \cdot x_i)^d$	Curved boundaries of degree d
RBF	$K(x, x_i) = \exp(-\gamma \lVert x - x_i \rVert^2)$	Default for non-linear data

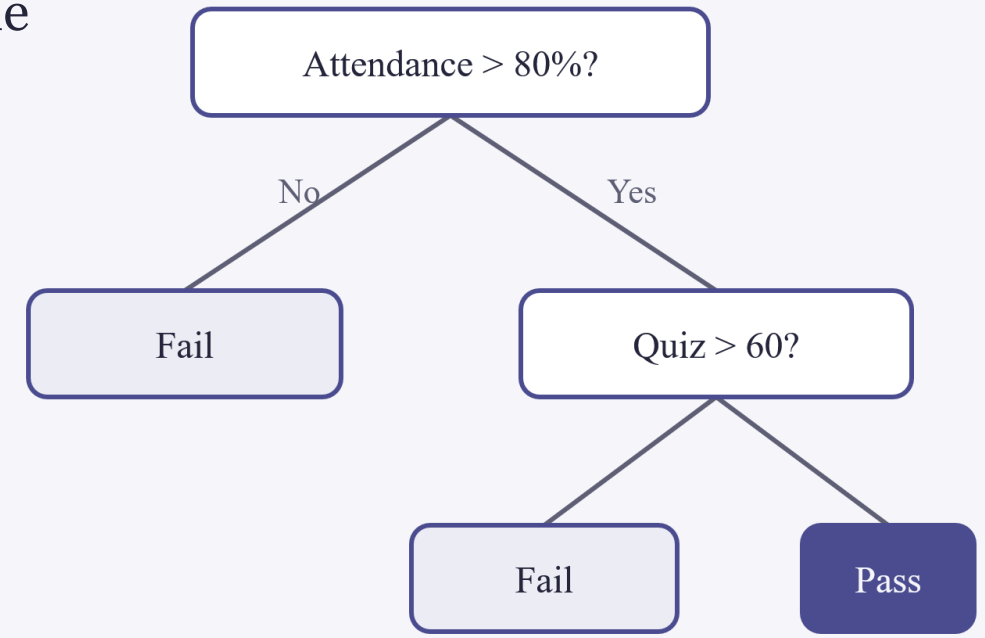
RBF gamma ($\gamma > 0$)	Effect
Small γ	Smooth, simple boundary: may underfit
Large γ	Very wiggly boundary: may overfit

Tune C and γ together with cross-validation. SVM is distance-based: scale the features.

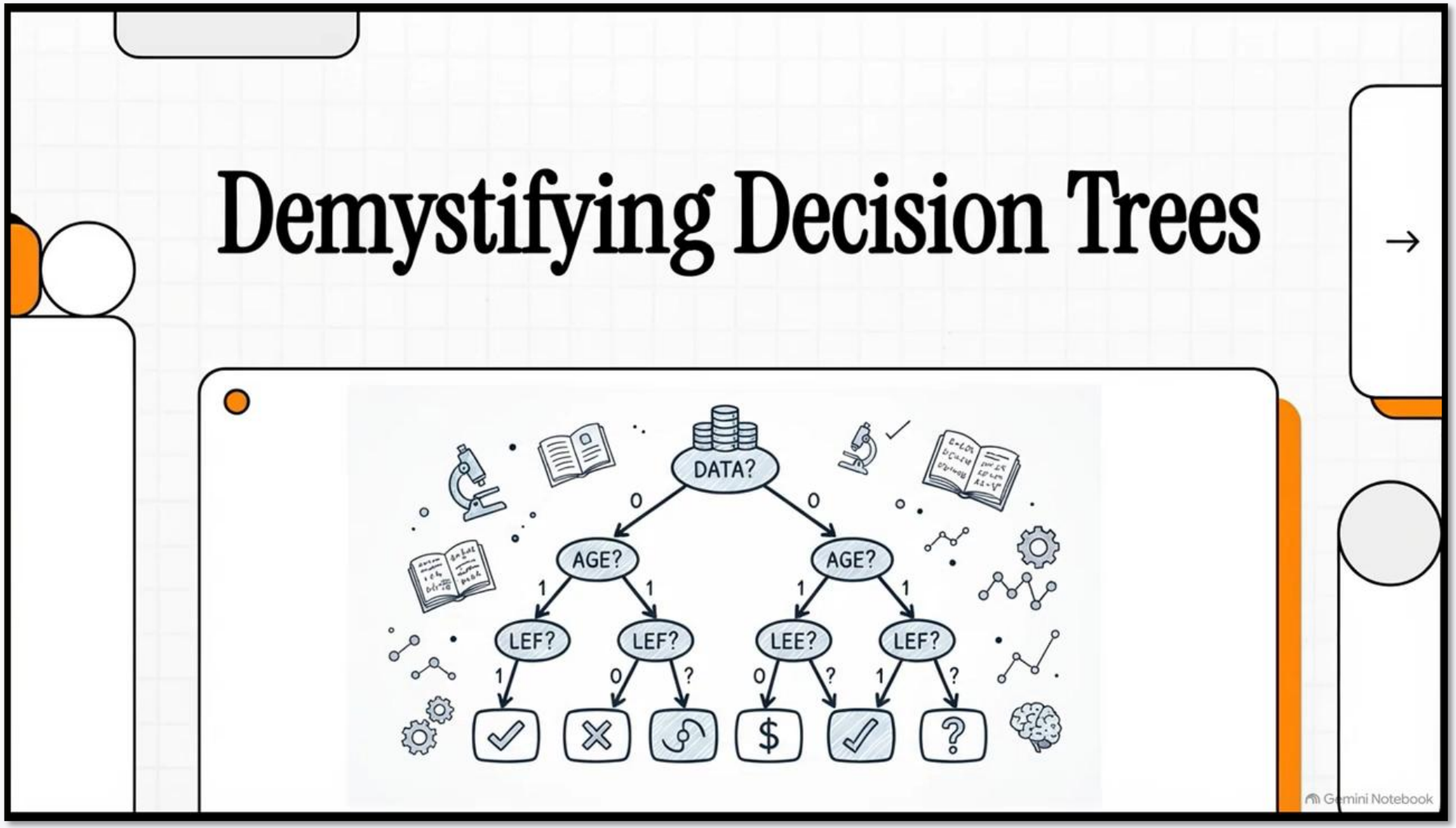
Decision Trees

Asks a series of yes/no questions on the features. Each split is chosen to make the groups as **pure** as possible (Gini impurity or entropy).

- Easy to explain: the rules can be read directly
- No scaling needed; handles mixed feature types
- A single deep tree overfits easily
- Key hyperparameters: max_depth, min_samples_leaf



Decision Trees Process



Part 2

Ensembles

Bagging, Random Forest and boosting

Combining many models: bagging vs boosting

Bagging (e.g. Random Forest)

- Train many trees **in parallel**, each on a random bootstrap sample
- Combine by majority vote
- Mainly **reduces variance** (overfitting)

Boosting (AdaBoost, Gradient Boosting, XGBoost)

- Train trees **one after another**
- Each new tree focuses on the previous errors
- Mainly **reduces bias**; can overfit if run too long

Ensembles are usually the strongest classical models for tabular data.

bagging vs boosting

Bagging vs. Boosting



Gemini Notebook

Random Forest

1. Draw a bootstrap sample (random rows, with replacement) for each tree
2. At each split, consider only a random subset of features
3. Grow many trees; each votes; the majority wins

Randomness makes the trees different, so their errors cancel out when they vote.

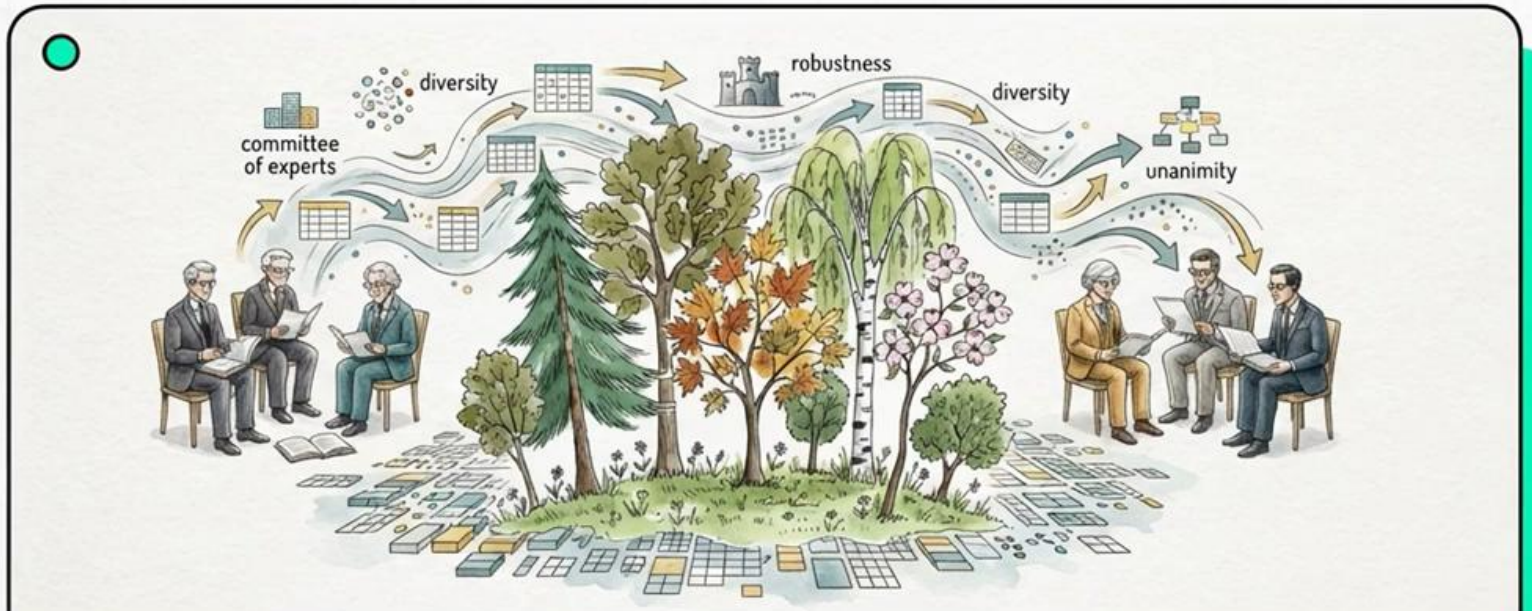
Feature importance shows which features drive predictions (used in the Week 4 lab and Assignments).

Hyperparameter	Typical values
n_estimators	100–500 trees
max_depth	None, 5, 10, 20
max_features	"sqrt", "log2"
min_samples_leaf	1, 2, 5

No scaling needed; robust default choice.

Random Forest Process

Random Forest Ensemble



Gemini Notebook

Boosting: AdaBoost, Gradient Boosting, XGBoost

Method	How each new tree learns	Notes
AdaBoost	Gives more weight to the samples the previous trees misclassified	Simple; sensitive to noisy labels
Gradient Boosting	Fits the remaining errors (residuals) of the current model	Strong accuracy; slower to train
XGBoost / LightGBM	Optimised gradient boosting with regularisation and fast training	Often top results on tabular data; handles missing values

Key hyperparameter	Effect
learning_rate	Smaller = slower but more careful learning (needs more trees)
n_estimators	Number of trees; too many can overfit
max_depth	Depth of each tree; usually small (3–6)

Choosing a classifier

Algorithm	Needs scaling	Interpretability	Speed	Good starting point for
Logistic Regression	Yes	High	Fast	Baseline; linear problems
Naïve Bayes	No	Medium	Very fast	Text; small datasets
KNN	Yes	Medium	Slow to predict	Small, low-dimensional data
SVM	Yes	Low (RBF)	Slow on large data	Medium-sized data, clear margins
Decision Tree	No	High	Fast	Explainable rules
Random Forest	No	Medium	Medium	Robust default for tabular data
Gradient Boosting / XGBoost	No	Medium	Medium	Highest accuracy on tabular data

No algorithm wins everywhere: compare several with cross-validation.

Part 3

Evaluating and selecting models

Confusion matrix, baselines, cross-validation and tuning

Confusion matrix and metrics

	Predicted positive	Predicted negative
Actual positive	TP: correct alarm	FN: missed case
Actual negative	FP: false alarm	TN: correct rejection

Accuracy = (TP + TN) / all

Precision = TP / (TP + FP)

Recall = TP / (TP + FN)

F1 = 2PR / (P + R)

False positive: actually negative but predicted positive. **False negative:** actually positive but predicted negative.

On imbalanced data, report recall, F1 and the confusion matrix, not accuracy alone.

Before any model

Start with a baseline.

A majority-class classifier (DummyClassifier) sets the minimum score. If a model cannot beat it clearly, it has learned little.

Comparing models with cross-validation

- Use **stratified k-fold** (k = 5 or 10) on the training data only
- Train and score each model on the same folds
- Report the **mean ± standard deviation**
- A difference smaller than the standard deviation may be due to chance

```
from sklearn.model_selection import
StratifiedKFold, cross_val_score
cv = StratifiedKFold(5, shuffle=True,
random_state=42)
for name, m in models.items():
    s = cross_val_score(m, X_train, y_train,
                        cv=cv,
scoring="f1_macro")
    print(name, s.mean().round(3),
s.std().round(3))
```

Hyperparameters to tune

Parameters are learned from data; **hyperparameters** are set before training and chosen by tuning.

Algorithm	Hyperparameters	Typical search values
Logistic Regression	C	0.01, 0.1, 1, 10
KNN	n_neighbors, weights	3, 5, 11, 21; uniform or distance
SVM	C, gamma, kernel	C: 0.1–100; gamma: 0.001–1; linear or rbf
Random Forest	n_estimators, max_depth, max_features	100–500; None, 10, 20; sqrt, log2
Gradient Boosting	learning_rate, n_estimators, max_depth	0.01–0.3; 100–500; 3–6

Grid search and random search

Grid search

Tries **every combination** of the values you list. Thorough, but slow when there are many hyperparameters.

Random search

Tries a **fixed number of random combinations**. Often finds good settings much faster.

- Put scaling and the model in one **Pipeline**, so each fold scales only its own training data
- Tune with cross-validation on the training set only
- Show the effect of each hyperparameter (e.g. a plot of score against C)
- Use the test set **once**, at the very end

Python: the full workflow

```
from sklearn.dummy import DummyClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.metrics import classification_report

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
cv = StratifiedKFold(5, shuffle=True, random_state=42)

base = DummyClassifier(strategy="most_frequent").fit(X_tr, y_tr) # 1. baseline
pipe = make_pipeline(StandardScaler(), SVC()) # 2. model
grid = {"svc__C": [0.1, 1, 10, 100], "svc__gamma": [0.001, 0.01, 0.1, 1]}
search = GridSearchCV(pipe, grid, cv=cv, scoring="f1_macro").fit(X_tr, y_tr) # 3-4. CV + tuning

print("Baseline:", base.score(X_te, y_te))
print("Best settings:", search.best_params_)
print(classification_report(y_te, search.predict(X_te))) # 5. test once
```

ML Classifier Guide



Gemini Notebook

Case study: banknote authentication

Is a banknote genuine or forged? Four features are extracted from wavelet-transformed images of each note: variance, skewness, kurtosis and entropy (UCI Banknote Authentication, 1,372 notes).

Step	What to do
Baseline	Majority class: about 56% accuracy (the larger class)
Candidates	Logistic Regression, KNN, SVM, Random Forest, Gradient Boosting
Compare	Stratified 5-fold cross-validation, macro F1, mean ± std
Tune	Grid search on the best two (e.g. SVM C and gamma; Random Forest depth and trees)
Test once	Confusion matrix: which error is worse here, a false positive or a false negative?

Applying this in the assignments

Assessment	What to use from this week
Assignment 1 (given week 2, due Week 5)	Two classifiers plus a majority-class baseline, compared with stratified cross-validation using default settings
Assignment 2 (given week 5, due Week 8)	A classical model on your own dataset, tuning at least 2 hyperparameters with cross-validation, and showing each one's effect
Group Project (given week 8, due Week 11)	Your Assignment 2 models become the baselines for the deep learning method

- **Key takeaways**
  - Logistic Regression, Naïve Bayes, KNN, SVM and Decision Trees classify in different ways; none is always best
  - Scale features for distance-based models (KNN, SVM) and Logistic Regression
  - Ensembles combine many trees: bagging reduces variance, boosting reduces bias
  - Always start with a baseline, and compare models with stratified cross-validation
  - Tune hyperparameters inside a Pipeline on training data; report mean ± std
  - Use the test set once, at the end, and look beyond accuracy